

OSDBMS-V-2026-T038

TEST PLAN

WALnut

A Concurrent Auction Engine with Deadlock Detection & Recovery

Team: The Acid Mechanics – T038

Vinayak Suyal · Sushmit Singh Rawat · Kanak Rawat · Suhavi Jugran

Version 1.0 · Companion to the WALnut PRD, Architecture Doc & API Spec
Status: Draft for Review

Table of Contents

- 1. Introduction
- 2. Test Scope
- 3. Test Strategy & Levels
- 4. Test Environment & Tools
- 5. Requirements Traceability Matrix
- 6. Test Cases — Storage Manager
- 7. Test Cases — Buffer Pool
- 8. Test Cases — Concurrency & Lock Manager
- 9. Test Cases — Deadlock Detection & Recovery
- 10. Test Cases — Write-Ahead Logging & Crash Recovery
- 11. Test Cases — API Layer
- 12. Test Cases — Dashboard (Manual / Exploratory)
- 13. Performance & Load Test Plan
- 14. Milestone Acceptance Gates
- 15. Defect Management
- 16. Testing Risks & Mitigations

1. Introduction

This Test Plan defines how WALnut's correctness, concurrency safety, deadlock handling, and crash recovery are verified. It translates the functional and non-functional requirements in the PRD, and the internal designs in the Architecture Document and API Specification, into concrete, repeatable test cases with pass/fail criteria.

Because WALnut's core claims are about concurrent correctness and failure recovery — properties that don't show up in a casual manual run — this plan leans heavily on deterministic, scripted tests (fixed thread interleavings, controlled fault injection) rather than ad hoc testing.

2. Test Scope

2.1 In Scope

- Storage manager: page format read/write correctness, allocation/free-space management.
- Buffer pool: pin/unpin semantics, replacement policy, write-ahead rule enforcement, hit/miss accounting.
- Lock manager: Strict 2PL correctness under concurrent placeBid transactions.
- Deadlock manager: wait-for graph cycle detection, victim selection, rollback, automatic retry.
- Recovery manager: WAL correctness, redo/undo on restart, checkpointing.
- REST API and SSE layer: request/response contracts, error codes, live event correctness.
- Dashboard: manual/exploratory verification that the bidding and systems-monitor views reflect live engine state.
- Performance: latency, throughput, buffer hit rate, and deadlock frequency under concurrent load.

2.2 Out of Scope

- Authentication/authorization testing — not implemented per PRD assumptions.
- Multi-machine or distributed failure scenarios — single-instance design.
- Disk-corruption recovery — explicitly out of scope per PRD §4.3; only process crash/restart is tested.
- Stretch-goal features (B+ Tree index, SCAN/C-SCAN simulation, full SQL subset) unless and until they are pulled into scope.

3. Test Strategy & Levels

Level	Purpose	Primary Owner(s)
Unit tests	Verify individual subsystem functions in isolation (e.g. page allocation, lock table operations).	Subsystem owner (PRD §11)
Integration tests	Verify correct interaction across subsystem boundaries (e.g. buffer pool ↔ storage manager ↔ WAL).	Pairs of adjacent owners
Concurrency / stress tests	Verify correctness under many simultaneous transactions — the project's central claim.	Sushmit (lock manager), whole team

Level	Purpose	Primary Owner(s)
Fault-injection tests	Verify recovery correctness by killing the process at controlled points.	Kanak (WAL/recovery)
API/contract tests	Verify each endpoint matches the API Specification, including error codes.	Suhavi (API layer)
System / acceptance tests	End-to-end scenarios matching PRD Section 13 acceptance criteria, run at each weekly gate.	Whole team
Exploratory / manual	Human-driven checks of the dashboard's live views during and after each milestone.	Whole team

4. Test Environment & Tools

Aspect	Detail
Build & unit testing	CMake + GoogleTest / Catch2, run locally and in CI on every push.
Concurrency testing	Custom C++ test harnesses spawning fixed numbers of threads issuing scripted operation sequences.
Fault injection	Test harness that starts the engine as a subprocess and sends SIGKILL at a chosen log/flush point, then restarts and verifies state.
API testing	curl / Postman collections and/or a lightweight HTTP test script against the API Specification.
Load testing	Scripted concurrent bidders (the “scripted bidder” client from PRD §4.3 / FR-7) ramped to target concurrency levels.
Environment	Single development/demonstration machine (Linux/macOS), matching the PRD's portability constraint — no distributed test infrastructure.

5. Requirements Traceability Matrix

Maps each PRD functional/non-functional requirement to the test cases that verify it (defined in Sections 6–13).

Requirement	Verified By
FR-1 createUser	TC-API-01
FR-2 createItem	TC-API-01
FR-3 placeBid	TC-LCK-01, TC-LCK-02, TC-API-03
FR-4 getHighestBid	TC-API-01
FR-5 listBids	TC-API-01
FR-6 closeAuction	TC-API-01, TC-API-02
FR-7 reserveBundle	TC-DLK-01, TC-DLK-02, TC-DLK-03
FR-8 Deadlock detection & resolution	TC-DLK-01 – TC-DLK-04
FR-9 WAL logging & crash recovery	TC-REC-01 – TC-REC-04

Requirement	Verified By
FR-10 Live monitoring dashboard	TC-UI-02, TC-API-04
FR-11 Interactive manual bidding (UI)	TC-UI-01, TC-API-03
NFR: Concurrency Safety	TC-LCK-01, TC-LCK-02, TC-PERF-01
NFR: Reliability	TC-REC-01 – TC-REC-04
NFR: Deadlock Handling	TC-DLK-01 – TC-DLK-04
NFR: Performance	TC-PERF-01 – TC-PERF-03
NFR: Observability	TC-UI-01, TC-UI-02, TC-API-04

6. Test Cases — Storage Manager

ID	Description	Steps	Expected Result
TC-STO-01	Page write/read round-trip	Write a page with known content; flush; read it back.	Read content is byte-identical to what was written.
TC-STO-02	Free-space reuse	Delete a record freeing space on a page; insert a new record that fits.	New record reuses the freed space rather than allocating a new page.
TC-STO-03	New page allocation	Fill a page to capacity; insert one more record for the same table.	A new page is allocated and linked; no data corruption on the full page.

7. Test Cases — Buffer Pool

ID	Description	Steps	Expected Result
TC-BUF-01	Pinned page not evicted	Pin a page; fill remaining frames to force eviction pressure.	The pinned page is never selected as an eviction victim.
TC-BUF-02	Replacement policy correctness	Access a known sequence of pages exceeding frame count.	Evicted page matches the expected Clock/LRU victim for that sequence.
TC-BUF-03	Write-ahead rule enforced	Modify a page; force its eviction.	The corresponding WAL record is durably written before the page flush occurs (verified by log/flush ordering instrumentation).
TC-BUF-04	Hit/miss accounting	Run a fixed access pattern with known repeat rate.	Reported hit/miss counters match the expected values exactly.

8. Test Cases — Concurrency & Lock Manager

ID	Description	Steps	Expected Result
TC-LCK-01	No lost update under concurrent bids	Launch N threads placing increasing bids on the same item concurrently.	Final current_bid equals the true maximum submitted amount; bid count in Bids equals N (no bid silently dropped).
TC-LCK-02	No dirty read of uncommitted state	Thread A begins placeBid but is paused before commit; Thread B calls getHighestBid.	Thread B never observes Thread A's uncommitted current_bid.
TC-LCK-03	Correct blocking/queueing	Thread A holds the lock on item X; Thread B requests item X.	Thread B blocks and is added to the wait queue; a corresponding lock.waiting event is emitted.
TC-LCK-04	Wake-up on release	Thread A releases the lock on item X while Thread B is queued.	Thread B is granted the lock and proceeds without manual intervention or missed wake-up.

9. Test Cases — Deadlock Detection & Recovery

ID	Description	Steps	Expected Result
TC-DLK-01	Genuine cycle detected	T1: reserveBundle([A,B]); T2 (concurrent): reserveBundle([B,A]).	A wait-for cycle forms and is detected within one detection interval; deadlock.detected event fires with both transaction IDs.
TC-DLK-02	Victim rollback releases locks	Trigger TC-DLK-01; observe victim selection.	Chosen victim's locks are fully released; the surviving transaction proceeds and completes.
TC-DLK-03	Automatic retry succeeds	Trigger TC-DLK-01 through to victim abort.	The aborted transaction is automatically retried and eventually completes successfully once locks are available.
TC-DLK-04	No false positive on non-cyclic wait	T1 waits on T2; T2 does not wait on anyone.	No deadlock is reported; T1 proceeds normally once T2 releases its lock.

10. Test Cases — Write-Ahead Logging & Crash Recovery

ID	Description	Steps	Expected Result
TC-REC-01	Redo after crash post-commit	Commit a transaction (WAL record written); kill the process before the dirty page is flushed; restart.	On restart, the redo pass reapplies the committed change; the bid is present and correct.
TC-REC-02	Undo of in-flight transaction	Kill the process mid-transaction, after some writes but before COMMIT is logged; restart.	On restart, the undo pass reverts all of that transaction's partial writes; no partial bid is visible.

ID	Description	Steps	Expected Result
TC-REC-03	Checkpoint reduces recovery scope	Take a checkpoint; commit further transactions; crash; restart.	Recovery only redoes log records after the checkpoint LSN; result is identical to a full scan.
TC-REC-04	Mixed committed/uncommitted crash	Commit transactions T1, T2; leave T3 in-flight; crash; restart.	T1 and T2's effects are present; T3's effects are fully absent. No cross-contamination between transactions.

11. Test Cases — API Layer

ID	Description	Steps	Expected Result
TC-API-01	Happy-path schema conformance	Call each endpoint in the API Specification with valid input.	Response status and body match the documented schema exactly.
TC-API-02	Error code mapping	Call each endpoint with each documented error condition (missing field, unknown ID, low bid, closed auction).	Returned HTTP status and error.code match the API Specification's error table.
TC-API-03	Single code path for manual vs. scripted bids	Submit one bid via the dashboard's Place Bid control and one via a script, to the same endpoint.	Both traverse identical engine code (verified via logging/tracing); no divergent "demo-only" logic exists.
TC-API-04	SSE event correctness	Place a bid and trigger a deadlock while subscribed to /stream.	Expected event sequence (lock.acquired → ... → lock.released, and lock.waiting → deadlock.detected → deadlock.resolved) is observed in order.

12. Test Cases — Dashboard (Manual / Exploratory)

These are human-executed checks, run at the Week 8 integration gate and spot-checked earlier as each view comes online.

ID	Description	Steps	Expected Result
TC-UI-01	Manual bid submission	Select a fixed user_id from the selector; click Place Bid on an open item.	Highest bid and bid history update within ~1 second, matching PRD NFR: Observability and FR-11.
TC-UI-02	Live systems-monitor view	Run the TC-DLK-01 scenario while watching the dashboard.	Lock table and wait-for graph visibly update in real time, showing the cycle forming and resolving.
TC-UI-03	User switching	Switch the selected user_id in the UI between two demo users and place bids as each.	No reload required; subsequent bids are correctly attributed to the newly selected user_id.

13. Performance & Load Test Plan

Benchmarks are run with the scripted-bidder client (PRD §4.3) at increasing concurrency levels, and reported in the final documentation (PRD Section 13, Week 8 gate).

Metric	Method	Reported As
Latency	Time from placeBid request to commit, measured client-side, across a fixed number of concurrent bidders.	p50 / p95 / p99 latency (ms) per concurrency level
Throughput	Committed transactions per second under sustained concurrent load.	TPS vs. concurrency-level curve
Buffer pool hit rate	Hit/miss counters (TC-BUF-04) sampled under the same load, at several buffer-frame counts.	Hit rate (%) vs. frame count, showing thrashing at low counts
Deadlock frequency	Count of deadlock.detected events per unit time under a reserveBundle-heavy contention workload.	Deadlocks per second vs. concurrency level

14. Milestone Acceptance Gates

Each gate below mirrors the PRD's Section 13 acceptance criteria and must pass before the corresponding week is considered complete.

Week	Gate	Required Passing Tests
5 (MVP)	Persistent engine safely processes concurrent bids under Strict 2PL.	TC-STO-01-03, TC-BUF-01-04, TC-LCK-01-04
6	Deadlock cycle reliably detected and resolved without manual intervention.	TC-DLK-01-04
7	Fault-injected crash recovery is correct.	TC-REC-01-04
8	API/dashboard live, benchmarks and documentation complete.	TC-API-01-04, TC-UI-01-03, TC-PERF-01-03

15. Defect Management

- All failing tests and observed bugs are logged as GitHub Issues, tagged with the owning subsystem (Storage, Buffer, Lock, Deadlock, Recovery, API, UI).
- Each issue records: failing test ID (if applicable), reproduction steps, expected vs. actual behavior.
- Concurrency and recovery bugs are treated as blocking for the current week's gate; UI/cosmetic issues are not.
- A fix is verified by re-running the originating test case, not just by manual spot-check.

16. Testing Risks & Mitigations

Risk	Mitigation
Concurrency bugs are non-deterministic and may not reproduce reliably.	Prefer deterministic thread-interleaving harnesses (fixed pause points) over relying on natural timing for TC-LCK and TC-DLK cases.
Fault injection may itself introduce test-harness bugs that mask real recovery bugs.	Cross-check recovered state against an independently computed expected state, not just “no crash on restart.”
Manual/exploratory UI tests are not repeatable evidence for grading.	Pair each manual UI test with an underlying automated API/SSE test (e.g. TC-UI-01 paired with TC-API-03/04) that proves the same behavior programmatically.
Limited time in Weeks 7–8 for full performance sweeps.	Fix a small, pre-agreed set of concurrency levels for the benchmark rather than an open-ended sweep.